

WashControl

Especificação do Sistema Computacional

Sistema SaaS de Gestão para Lava-Cars e Estéticas Automotivas

Curso: Bacharelado em Ciência da Computação (BCC)

Disciplina: Modelagem de Sistemas Computacionais

Projeto: Análise e Projeto de Sistema Computacional

Condução: Leonardo dos Santos Marques

Versão: 1.0.0 **Data:** 17/04/2026

Histórico de Revisão

Data	Versão	Descrição	Autor
17/04/2026	1.0.0	Elaboração inicial do documento de análise e projeto	Leonardo dos Santos Marques

Sumário

- Introdução
 - Contexto de Negócio
 - Justificativa do Projeto
 - Benefícios
- Requisitos
 - Requisitos Funcionais

2. Requisitos Não Funcionais (FURPS+)

3. Modelagem do Sistema

1. Diagrama de Caso de Uso
2. Especificações de Casos de Uso
3. Modelo de Domínio
4. Diagrama de Classes
5. Diagrama de Objetos
6. Diagramas de Sequência

4. Diagramas de Atividade

5. Diagramas de Máquina de Estados

6. Protótipo

7. Item de Criatividade

8. Responsabilidades

1. Introdução

1.1 Contexto de Negócio

O mercado de cuidados automotivos — popularmente chamado de *detailing* — tem crescido de forma expressiva no Brasil. As pessoas investem cada vez mais em seus veículos e buscam serviços de qualidade. No entanto, a maioria dos estabelecimentos de pequeno e médio porte ainda opera de forma desorganizada: filas sem controle, anotações em papel, comunicação dispersa via WhatsApp e ausência de visibilidade financeira.

O **WashControl** é um sistema SaaS (*Software as a Service*) multi-tenant desenvolvido para atender lava-cars e estéticas automotivas. Cada estabelecimento cadastrado na plataforma é tratado como um *tenant* isolado, com seus próprios clientes, veículos, funcionários, ordens de serviço e dados financeiros. A plataforma é administrada centralmente por um Super Admin, que gerencia os tenants, planos de assinatura e faturamento.

A arquitetura multi-tenant permite que o WashControl atenda simultaneamente dezenas de estabelecimentos com um único sistema, reduzindo custos de infraestrutura e facilitando a escalabilidade do produto.

1.2 Justificativa do Projeto

O principal problema que o WashControl resolve é a **falta de previsibilidade e controle** nos lava-cars. Quando um cliente deixa o veículo e recebe a informação "volta em 2 horas", mas ao chegar o serviço nem começou, a confiança é destruída. Da mesma forma, o dono do estabelecimento frequentemente não sabe com precisão quanto lucrou no dia, quais funcionários são mais produtivos ou quais insumos estão em falta.

O projeto se justifica pela necessidade de **automatizar o fluxo operacional** — desde o agendamento até a entrega do veículo — e fornecer **visibilidade financeira em tempo real** ao gestor. Estabelecimentos que utilizam sistemas organizados passam mais credibilidade ao cliente, conseguem atender maior volume de serviços sem perda de qualidade e tomam decisões baseadas em dados.

Do ponto de vista de mercado, trata-se de uma oportunidade de negócio B2B (empresa-para-empresa) com modelo de receita recorrente via assinatura (SaaS), com baixa concorrência de sistemas especializados acessíveis para pequenos e médios estabelecimentos.

1.3 Benefícios

Para o dono do negócio (Manager):

- Controle financeiro real com DRE automática (receitas × despesas)

- Visibilidade da produtividade da equipe e cálculo automático de comissões
- Gestão de estoque integrada ao fluxo de atendimento
- Programa de fidelidade configurável para reter clientes

Para o lavador (Washer):

- Interface simplificada e responsiva para uso no pátio via celular
- Fila de serviços organizada, eliminando conflitos de atribuição

Para o cliente:

- Agendamento prévio de horário
- Notificação automática quando o veículo estiver pronto (RF09)
- Histórico de atendimentos e benefícios do programa de fidelidade

Para a operação como um todo:

- Redução de erros humanos (dupla marcação, serviços não cobrados)
- Rastreabilidade completa do ciclo de vida de cada ordem de serviço
- Isolamento de dados entre estabelecimentos (multi-tenancy seguro)

2. Requisitos

2.1 Requisitos Funcionais

ID	Descrição	Requisitante	Detalhes
RF01	O sistema deve permitir o cadastro e gerenciamento de Tenants (lavajatos parceiros).	SUPER_ADMIN	Envolve a criação da conta principal da loja no modelo SaaS, com nome, slug único e plano de assinatura.
RF02	O sistema deve permitir o cadastro de clientes e seus respectivos veículos.	MANAGER	Deve suportar a vinculação de múltiplos veículos a um único Customer. A placa do veículo deve ser única por tenant.
RF03	O sistema deve gerenciar o ciclo de vida da Ordem de Serviço (OS) através de um Kanban.	MANAGER	A OS deve transitar pelos status: PENDING, WAITING_QUEUE, IN_PROGRESS, READY, COMPLETED ou CANCELED.
RF04	O sistema deve permitir a visualização e atualização de status das OSs ativas no pátio.	WASHER	Interface simplificada para que o lavador mova o card da OS (ex: de WAITING_QUEUE para IN_PROGRESS).
RF05	O sistema deve permitir o agendamento prévio (Appointment) de serviços de lavagem.	Customer / MANAGER	O cliente final ou o gerente podem reservar um horário na agenda. Um agendamento pode gerar no máximo uma OS (idempotência).
RF06	O sistema deve abater automaticamente produtos do estoque (InventoryTransaction) ao concluir uma OS.	MANAGER	Integração entre a conclusão da OS e a baixa dos insumos utilizados nos itens da OS.
RF07	O sistema deve gerar uma transação financeira de receita (FinancialTransaction) ao concluir uma OS.	MANAGER	O fechamento da OS no Kanban engatilha o registro financeiro automático com método de pagamento e valor total.
RF08	O sistema deve permitir o registro e controle de despesas fixas (FixedExpense) e pagamentos de funcionários.	MANAGER	Controle de custos operacionais como água, luz, aluguel e comissões calculadas sobre OSs concluídas.
RF09		Customer	

	O sistema deve disparar notificações sobre a mudança de status do veículo.		O cliente deve ser avisado (ex: via WhatsApp ou Push) quando o status mudar para READY.
RF10	O sistema deve prover relatórios gerenciais sobre o volume de OSs, receitas e despesas.	MANAGER	Dashboard unificando os dados financeiros e operacionais do Tenant (DRE, fluxo de caixa, KPIs).

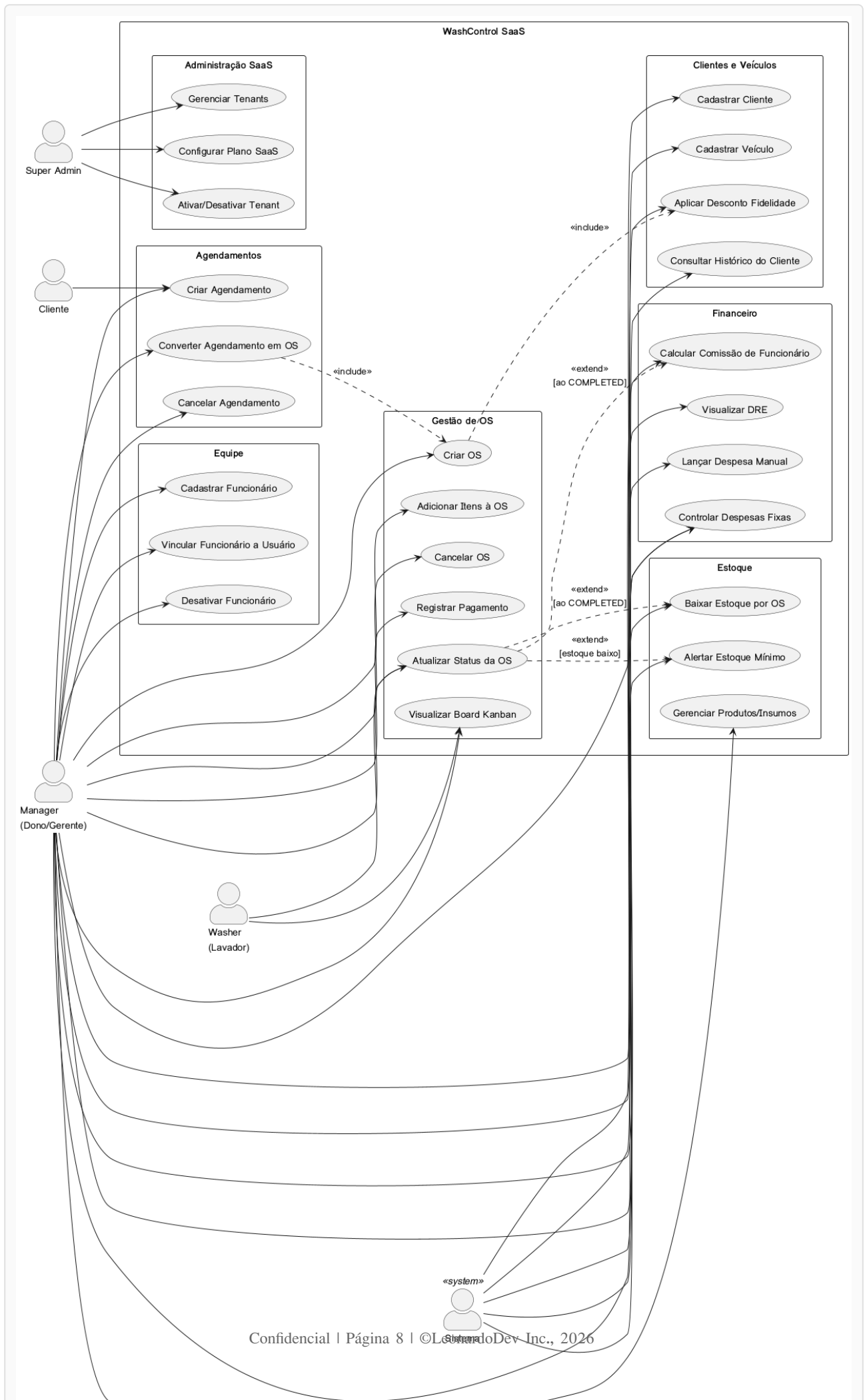
2.2 Requisitos Não Funcionais (Classificação FURPS+)

ID	Descrição	Requisitante	Categoria FURPS+	Detalhes
RNF01	O sistema deve possuir uma interface responsiva para adaptação a telas de smartphones.	WASHER	Usability	Essencial para o uso do sistema no pátio do lava-jato via celular, com mãos potencialmente molhadas.
RNF02	O sistema deve garantir o isolamento lógico dos dados entre diferentes lojas.	SUPER_ADMIN	Reliability	Sendo multi-tenant, um gerente nunca pode acessar dados de outro tenant. O tenantId é obrigatório em todas as queries.
RNF03	A atualização de status da OS no painel Kanban deve ser refletida na interface em no máximo 2 segundos.	MANAGER	Performance	O tempo de resposta deve garantir a fluidez da operação no pátio em tempo real.
RNF04	O sistema deve utilizar o padrão Repository com ORM (Prisma) para o acesso a dados.	Equipe Dev	Supportability	Facilita a manutenção do código, trocas de SGBD e escalabilidade do sistema.
RNF05	O sistema deve restringir o acesso a funcionalidades financeiras e de estoque ao papel de gerência.	MANAGER	+ Security	O WASHER só tem acesso operacional (pátio, OS), enquanto o MANAGER tem acesso total à sua loja.
RNF06	O modelo de dados central deve ser armazenado em banco de dados relacional.	Equipe Arq.	+ Implementation	Restrição tecnológica atendida pelo uso do schema.prisma com PostgreSQL.

3. Modelagem do Sistema

3.1 Diagrama de Caso de Uso

O diagrama abaixo apresenta os principais atores do WashControl — Manager, Washer, Cliente e Super Admin — e suas interações com os módulos do sistema. As relações `<<include>>` e `<<extend>>` destacam as automações que o sistema executa de forma transparente ao usuário.



3.2 Especificações de Casos de Uso

UC-01 — Criar Ordem de Serviço

Breve Descrição: Permite que o Manager registre a entrada de um veículo no sistema, criando uma Ordem de Serviço vinculada a um cliente, um veículo e com os itens de serviço/insumos a serem executados. A OS é criada com status PENDING e passa a compor o board Kanban.

Ator Principal: Manager | **Ator Secundário:** Sistema (valida dados, persiste no banco, verifica fidelidade)

Precondições:

- O Manager deve estar autenticado no sistema com papel MANAGER.
- O cliente deve estar previamente cadastrado no sistema (ou ser cadastrado durante o fluxo).
- O veículo deve estar vinculado ao cliente.
- Deve haver pelo menos um item (serviço ou insumo) para compor a OS.

Pós-condições:

- Uma nova OS é criada com status PENDING no banco de dados.
- A OS aparece na coluna PENDING do board Kanban.
- Os itens (OrderItems) estão vinculados à OS com preços e quantidades.

Fluxo Básico:

1. O Manager acessa o módulo `/dashboard/os` e clica em "Nova OS".
2. O sistema exibe o formulário de criação com campos: cliente, veículo, itens, observações e pagamento antecipado.
3. O Manager seleciona o cliente pelo nome ou telefone.
4. O sistema lista os veículos vinculados ao cliente selecionado.
5. O Manager seleciona o veículo a ser atendido.
6. O Manager adiciona os itens da OS (serviços e/ou insumos) com quantidades e preços unitários.
7. O sistema calcula e exibe o valor total em tempo real.
8. O Manager confirma a criação clicando em "Criar OS".
9. O sistema persiste a OS com status PENDING e redireciona para o board Kanban.
10. O sistema verifica se o cliente atingiu o intervalo de fidelidade
(`totalWashes % loyaltyInterval == 0`) e, se aplicável, aplica desconto automaticamente.

Fluxos Alternativos:

- *Cliente não cadastrado:* O Manager não encontra o cliente na busca → Clica em "Novo Cliente" → O sistema exibe o formulário de cadastro → O Manager preenche e confirma → O fluxo retorna ao passo 4.
- *Criação via Agendamento:* O Manager acessa `/dashboard/agendamentos` → Seleciona um agendamento com status SCHEDULED → Clica em "Converter em OS" → O sistema preenche automaticamente cliente e veículo → O fluxo continua a partir do passo 6. O sistema impede a criação de uma segunda OS para o mesmo agendamento (idempotência via `Appointment.orderId @unique`).

Fluxos de Exceção:

- *Veículo sem itens:* O Manager tenta confirmar a OS sem nenhum item adicionado → O sistema exibe: "A OS deve conter pelo menos um serviço ou insumo" → O fluxo retorna ao passo 6.

UC-03 — Atualizar Status da Ordem de Serviço

Breve Descrição: Permite que o Manager ou o Washer movam a OS entre os estados do ciclo de vida (PENDING → WAITING_QUEUE → IN_PROGRESS → READY → COMPLETED). Ao atingir o status COMPLETED, o sistema executa automaticamente os processos financeiro, de estoque e de fidelidade.

Ator Principal: Manager (para todos os status) / Washer (apenas IN_PROGRESS e READY) |

Ator Secundário: Sistema

Precondições:

- O usuário deve estar autenticado.
- A OS deve existir e pertencer ao tenant do usuário.
- A transição de status deve ser válida conforme o ciclo de vida definido.

Pós-condições:

- O status da OS é atualizado no banco de dados.
- Os campos de timestamp correspondentes são preenchidos (startedAt, finishedAt, completedAt).
- Se o novo status for COMPLETED: FinancialTransaction (INCOME) criada, InventoryTransactions OUT criadas, estoque decrementado, `Customer.totalWashes` incrementado e comissão calculada.
- A interface do Kanban é atualizada em no máximo 2 segundos (RNF03).

Fluxo Básico — Conclusão de OS (READY → COMPLETED):

1. O Manager acessa o board Kanban em `/dashboard/os`.

2. O Manager localiza a OS na coluna READY e clica em "Concluir / Registrar Pagamento".
3. O sistema exibe o formulário de conclusão: método de pagamento, valor total e valor adiantado.
4. O Manager confirma o método de pagamento (PIX, Dinheiro, Cartão Crédito, Cartão Débito).
5. O Manager clica em "Confirmar Conclusão".
6. O sistema atualiza o status para COMPLETED e registra `completedAt = now()`.
7. O sistema cria automaticamente a `FinancialTransaction` (INCOME), as `InventoryTransaction` OUT para cada insumo e decrementa o `Product.stock`.
8. O sistema incrementa `Customer.totalWashes`, verifica a fidelidade e calcula a comissão do Employee vinculado.
9. O card da OS desaparece do Kanban e aparece no histórico.

Fluxos Alternativos:

- *Lavador inicia serviço (WAITING_QUEUE → IN_PROGRESS):* O Washer acessa `/dashboard/patio` → Visualiza as OSs em WAITING_QUEUE → Clica em "Iniciar" → O sistema atualiza para IN_PROGRESS e registra `startedAt = now()`.
- *Cancelamento:* O Manager seleciona "Cancelar OS" → O sistema solicita confirmação e motivo → O Manager confirma → O sistema atualiza para CANCELED. Somente MANAGER pode cancelar (RNF05).

Fluxos de Exceção:

- *Transição de status inválida:* O sistema rejeita transições fora do ciclo definido (ex: PENDING → COMPLETED) e exibe mensagem de erro informando os status válidos.

UC-11 — Criar Agendamento

Breve Descrição: Permite que o Manager registre previamente um horário de atendimento para um cliente e veículo, reservando uma vaga na agenda do lava-car. O agendamento permanece com status SCHEDULED até ser convertido em OS ou cancelado.

Ator Principal: Manager | **Ator Secundário:** Sistema (valida disponibilidade de horário, envia notificação futura ao cliente)

Precondições:

- O Manager deve estar autenticado com papel MANAGER.
- O cliente e veículo devem estar cadastrados no sistema.
- O horário solicitado não deve estar ocupado por outro agendamento.

Pós-condições:

- Um novo Appointment é criado com status SCHEDULED.

- O agendamento aparece no calendário do sistema.
- O cliente pode receber uma notificação de confirmação (RF09).

Fluxo Básico:

1. O Manager acessa o módulo `/dashboard/agendamentos` e clica em "Novo Agendamento".
2. O sistema exibe o formulário: data/hora, cliente, veículo, serviço desejado e observações.
3. O Manager seleciona a data e horário desejados.
4. O sistema verifica a disponibilidade do horário selecionado.
5. O Manager seleciona o cliente e o veículo.
6. O Manager seleciona o tipo de serviço (opcional) e confirma o agendamento.
7. O sistema persiste o Appointment com status SCHEDULED e o exibe no calendário.

Fluxos Alternativos:

- *Conversão em OS no dia do atendimento:* O Manager localiza o agendamento → Clica em "Converter em OS" → O sistema cria uma OS pré-preenchida → O Manager adiciona os itens finais e confirma → O sistema vincula a OS ao agendamento via `Appointment.orderId` → O status do agendamento é atualizado para COMPLETED.
- *Cancelamento do Agendamento:* O Manager localiza o agendamento → Clica em "Cancelar Agendamento" → O sistema solicita confirmação → O status é atualizado para CANCELED.

Fluxos de Exceção:

- *Conflito de horário:* O sistema detecta outro agendamento no mesmo horário → Exibe alerta: "Horário indisponível. Selecione outro horário." → O Manager seleciona um horário alternativo.
- *Tentativa de segunda OS para o mesmo agendamento:* O Manager tenta converter um agendamento que já possui uma OS vinculada → O sistema rejeita a operação exibindo: "Este agendamento já possui uma OS vinculada" → O sistema redireciona para a OS existente (idempotência garantida por `Appointment.orderId @unique`).

3.3 Modelo de Domínio

O modelo de domínio apresenta as entidades do problema de negócio, seus atributos essenciais, multiplicidades e relacionamentos conceituais, sem foco em detalhes de implementação técnica.

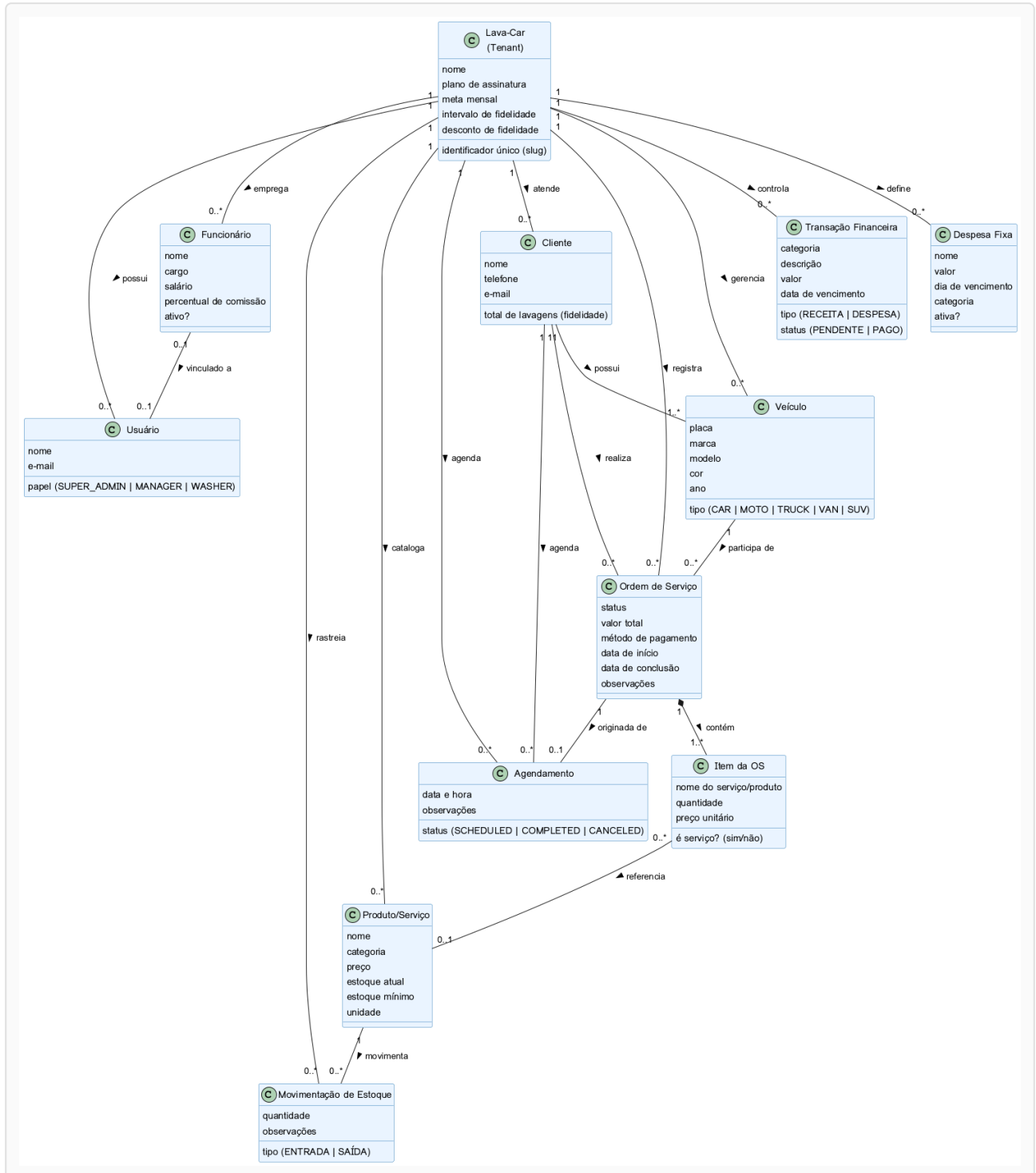


Figura 2 — Modelo de Domínio do WashControl

3.4 Diagrama de Classes

O diagrama de classes detalha a estrutura do sistema, incluindo atributos tipados, métodos e os relacionamentos entre as classes. A composição com `Tenant` como raiz de todas as entidades reflete a arquitetura multi-tenant do sistema.



Figura 3 — Diagrama de Classes do WashControl

3.5 Diagrama de Objetos

O diagrama de objetos apresenta uma instância concreta do sistema, representando um cenário real de atendimento: o cliente **João Silva** deixou seu veículo **Honda Civic (ABC-1234)** para uma lavagem completa com cera. Este snapshot representa o momento em que a OS está com status **IN_PROGRESS**.

Justificativa da escolha das classes: As classes Order, Customer, Vehicle, OrderItem e Product foram selecionadas por constituírem o núcleo transacional do sistema — são as entidades que trocam estado com maior frequência durante a operação diária. O Tenant foi incluído para ilustrar o isolamento multi-tenant. Employee e FinancialTransaction foram omitidos pois ainda não há valores definitivos neste estado (a OS está em andamento, não concluída).

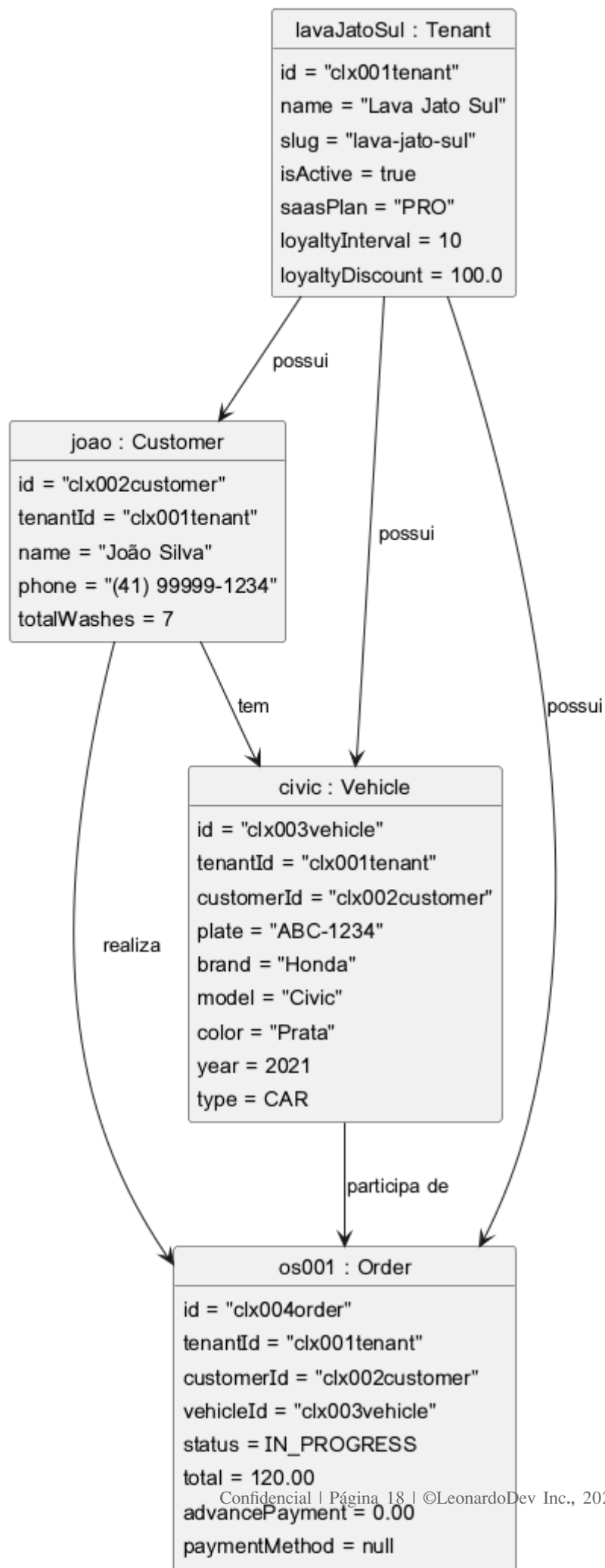


Figura 4 — Diagrama de Objetos: instância de atendimento em andamento

3.6 Diagramas de Sequência

Os cinco diagramas de sequência a seguir representam as principais interações do sistema, contemplando todos os atores e as trocas de mensagens entre eles e os objetos do sistema. Fragmentos de interação (alt, loop, opt) são utilizados para enriquecer a semântica dos diagramas.

DS-01 — Autenticação e Acesso ao Dashboard

Este diagrama descreve o fluxo de autenticação do usuário, desde o preenchimento do formulário de login até o redirecionamento ao dashboard, incluindo os casos de senha incorreta e usuário não encontrado.

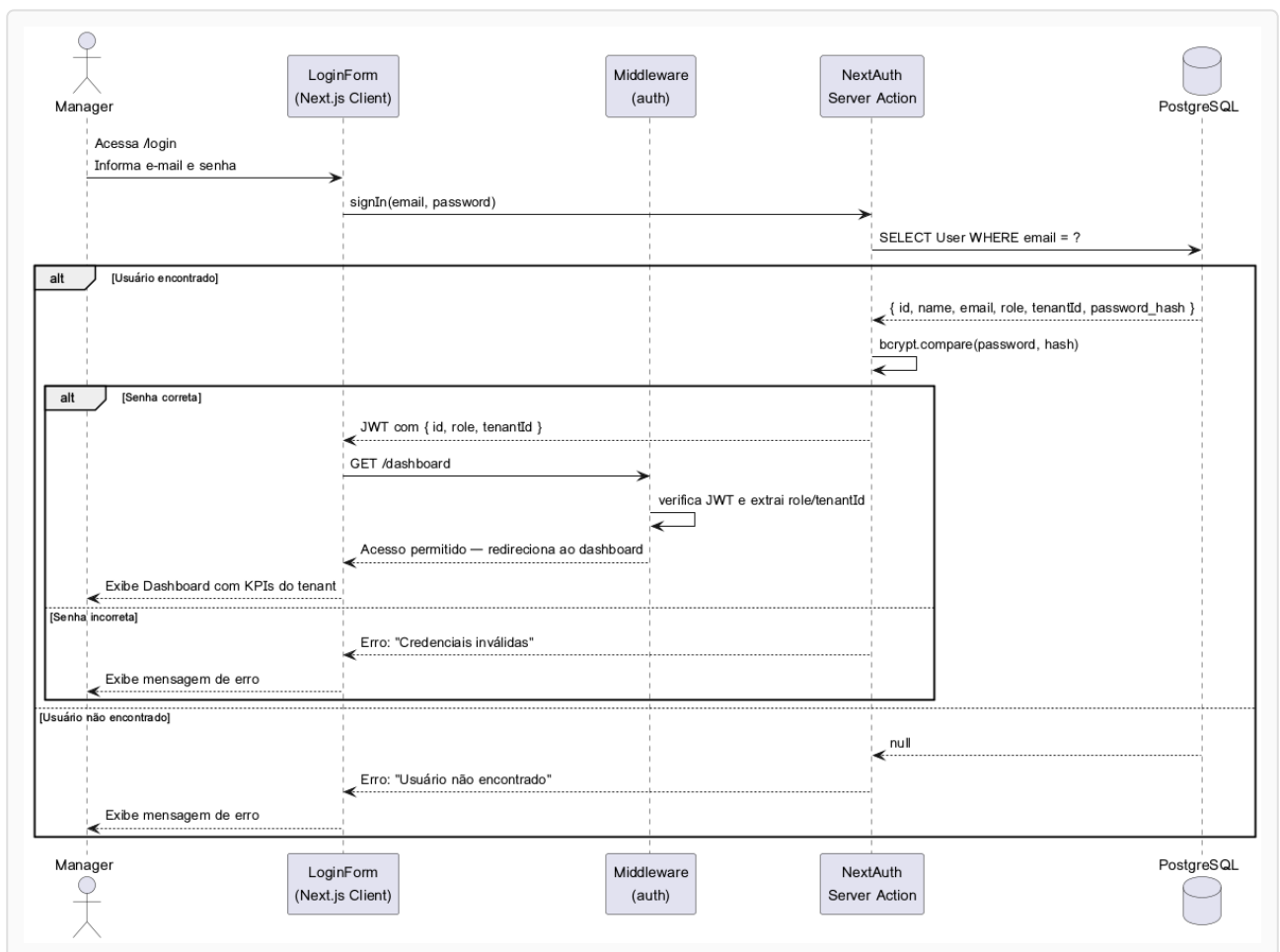


Figura 5 — DS-01: Autenticação e Acesso ao Dashboard

DS-02 — Criar e Concluir Ordem de Serviço

Este diagrama representa o ciclo completo de uma OS: criação pelo Manager, movimentação pelo Washer no pátio e conclusão com registro automático de transação financeira, baixa de estoque e atualização de fidelidade.

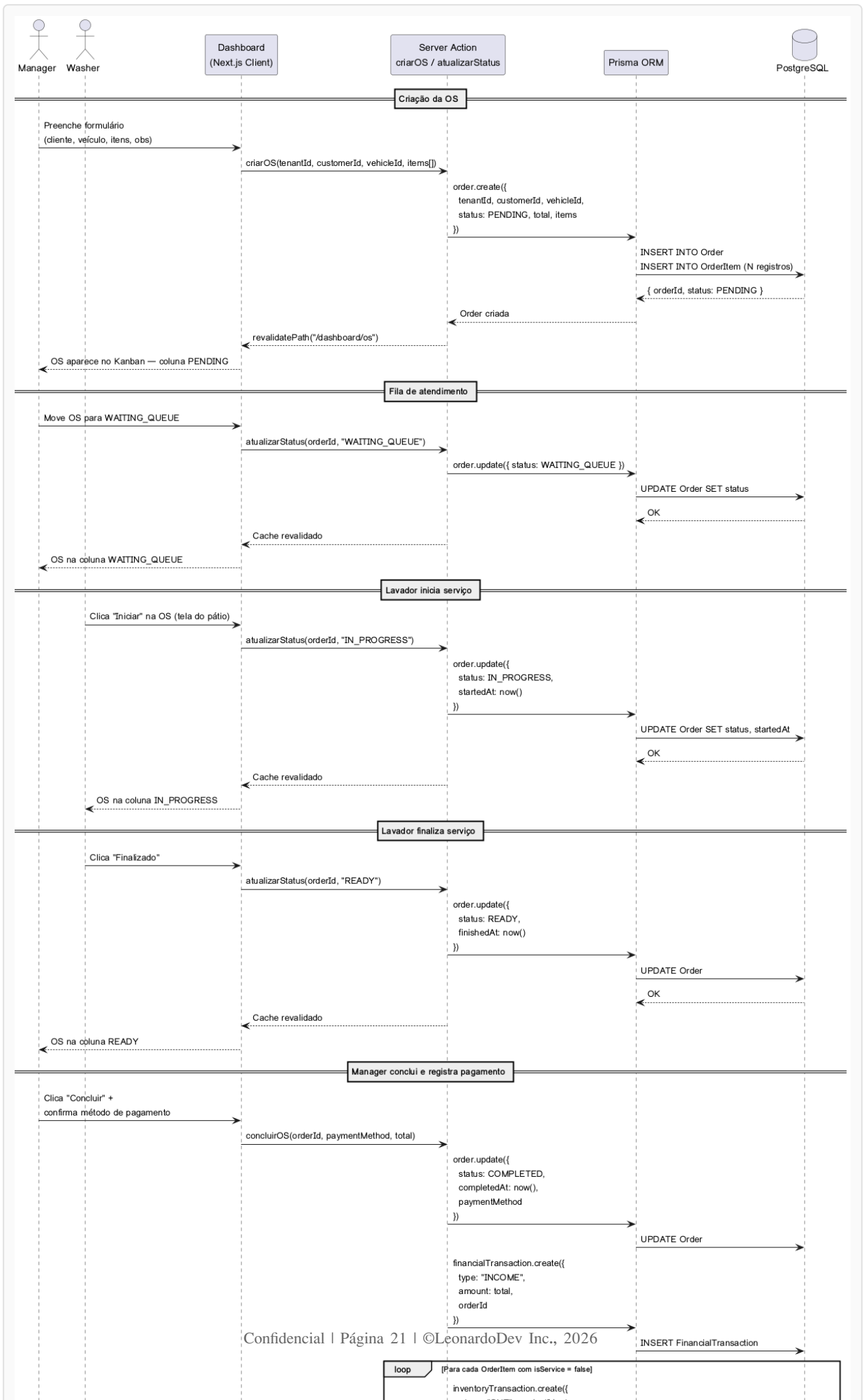


Figura 6 — DS-02: Ciclo Completo da Ordem de Serviço

DS-03 — Criar Agendamento e Converter em OS

Este diagrama descreve o fluxo de criação de um agendamento e sua posterior conversão em Ordem de Serviço, incluindo a proteção de idempotência que impede a criação de uma segunda OS para o mesmo agendamento.

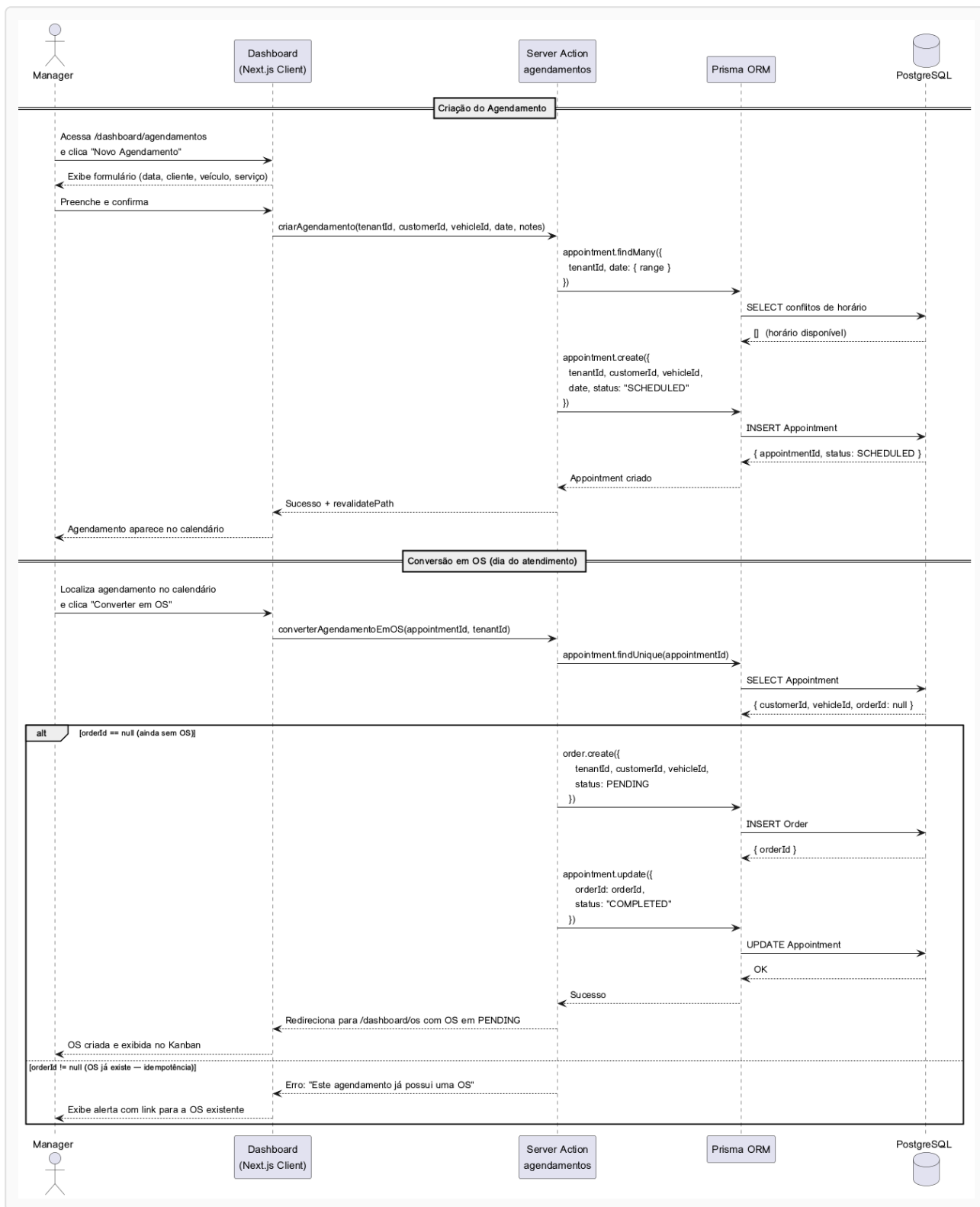


Figura 7 — DS-03: Agendamento e Conversão em OS

DS-04 — Baixa Automática de Estoque ao Concluir OS

Este diagrama detalha o processo de baixa de estoque executado automaticamente ao concluir uma OS, incluindo o tratamento de estoque insuficiente e o alerta de estoque mínimo.

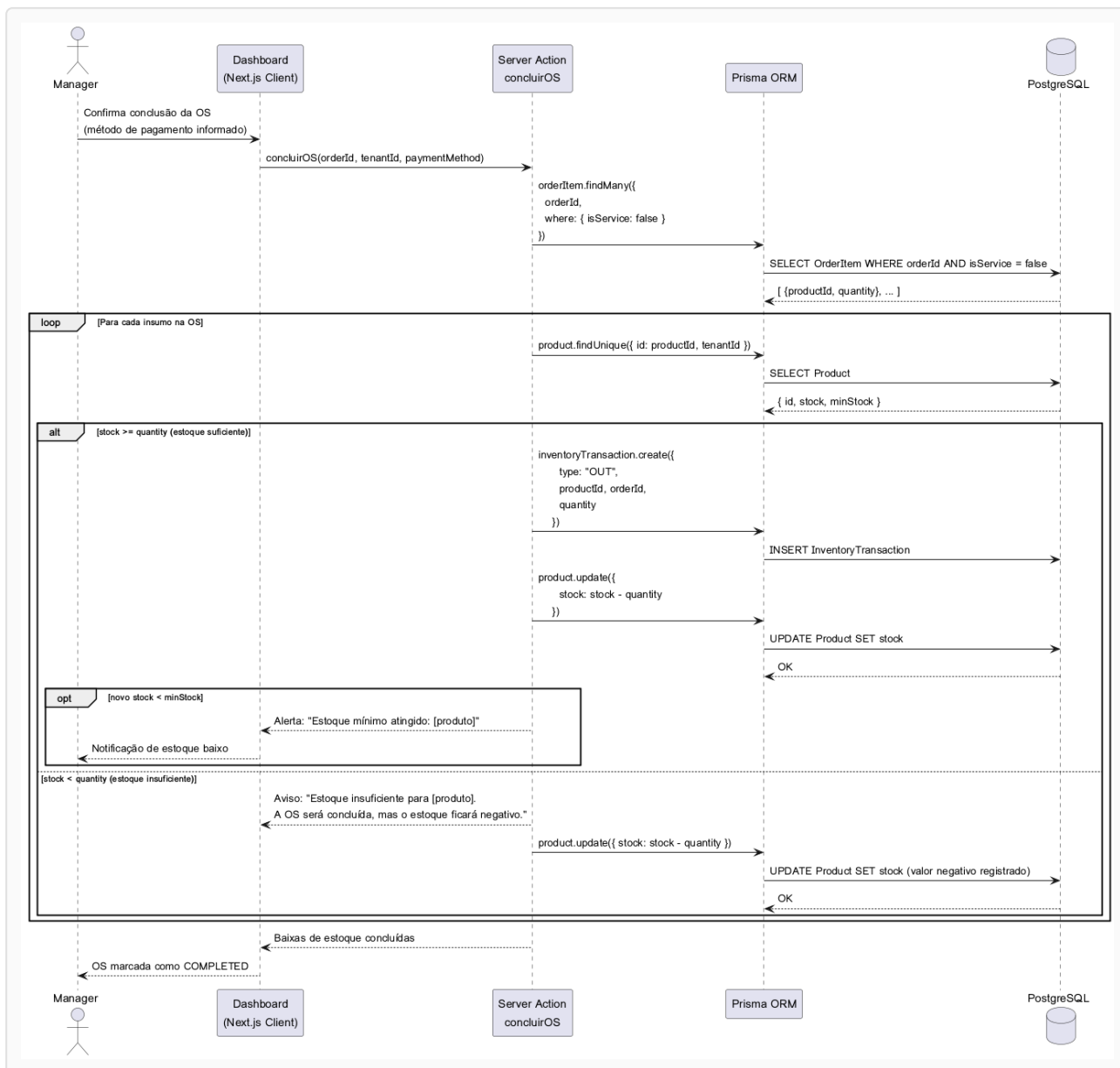


Figura 8 — DS-04: Baixa Automática de Estoque

DS-05 — Visualização do Relatório Financeiro (DRE)

Este diagrama descreve o fluxo de carregamento do módulo financeiro, com a consulta de receitas, despesas e despesas fixas, e o cálculo da DRE mensal.

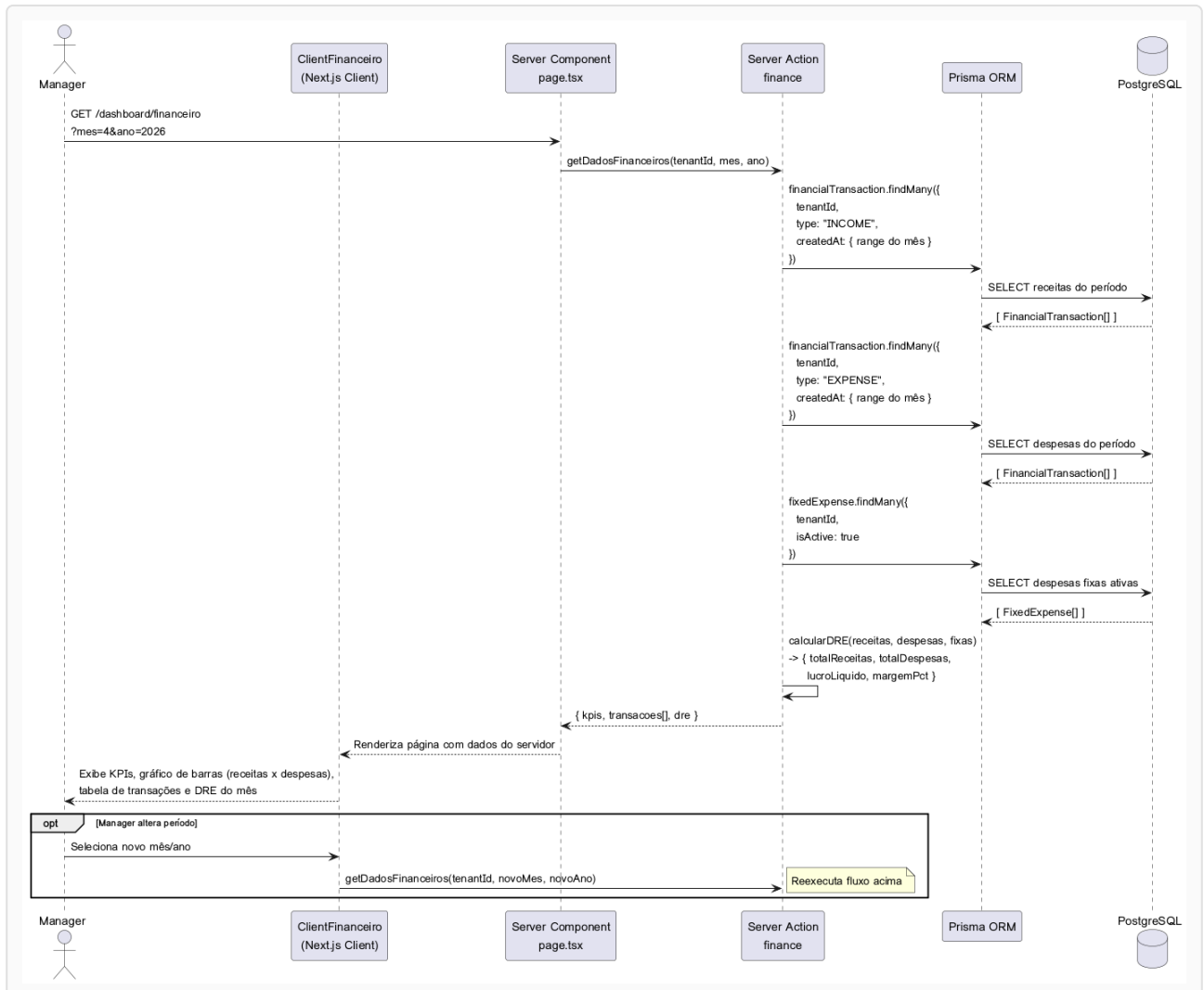


Figura 9 — DS-05: Visualização do Relatório Financeiro (DRE)

4. Diagramas de Atividade

Os diagramas de atividade a seguir especificam os principais fluxos de processo do sistema, com construções de paralelismo (fork/join) e condições de decisão.

DA-01 — Fluxo Completo de Atendimento (Ciclo da OS)

Este diagrama representa o processo completo de atendimento, desde a chegada do veículo até a conclusão da OS, envolvendo os atores Manager e Washer e as automações do sistema em paralelo.

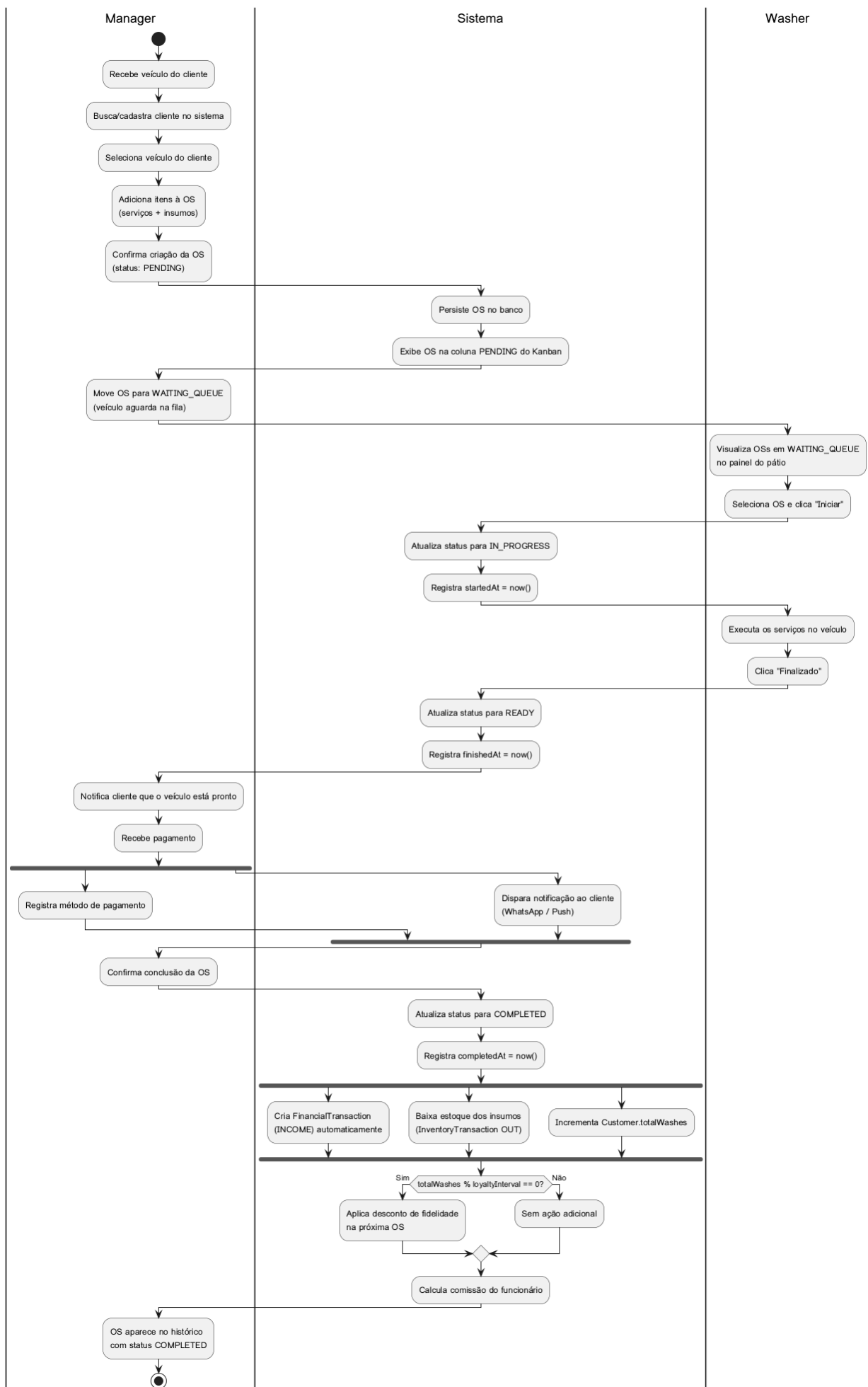


Figura 10 — DA-01: Fluxo Completo de Atendimento

DA-02 — Fluxo de Agendamento

Este diagrama especifica o processo de criação e gestão de agendamentos, incluindo a verificação de disponibilidade, a conversão em OS e o tratamento de não comparecimento do cliente.

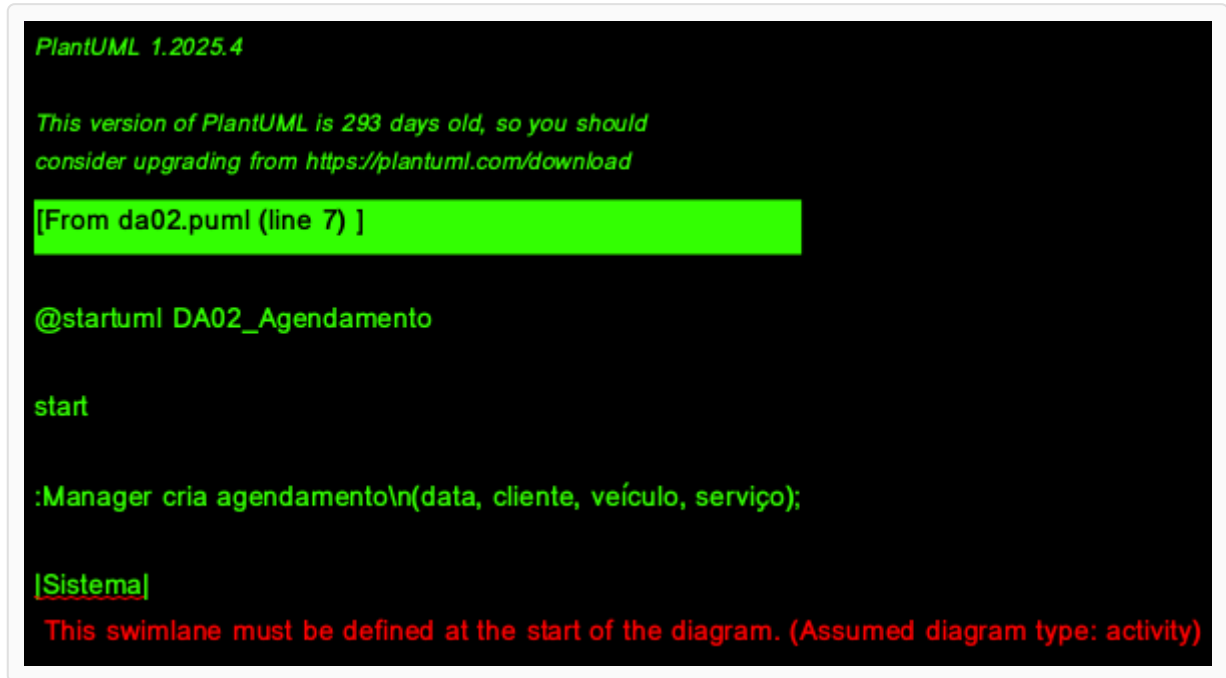


Figura 11 — DA-02: Fluxo de Agendamento

DA-03 — Cálculo de Fidelidade e Desconto

Este diagrama detalha o processo de verificação e aplicação do programa de fidelidade, executado automaticamente ao concluir uma OS.

PlantUML 1.2025.4

This version of PlantUML is 293 days old, so you should consider upgrading from <https://plantuml.com/download>

[From da03.puml (line 7)]

@startuml DA03_Fidelidade

start

:OS é concluída\n(status -> COMPLETED);

|Sistema|

This swimlane must be defined at the start of the diagram. (Assumed diagram type: activity)

Figura 12 — DA-03: Cálculo de Fidelidade e Desconto

DA-04 — Lançamento de Despesa Fixa Mensal

Este diagrama representa o processo automatizado de lançamento de despesas fixas, com verificação de idempotência para evitar lançamentos duplicados no mesmo mês.

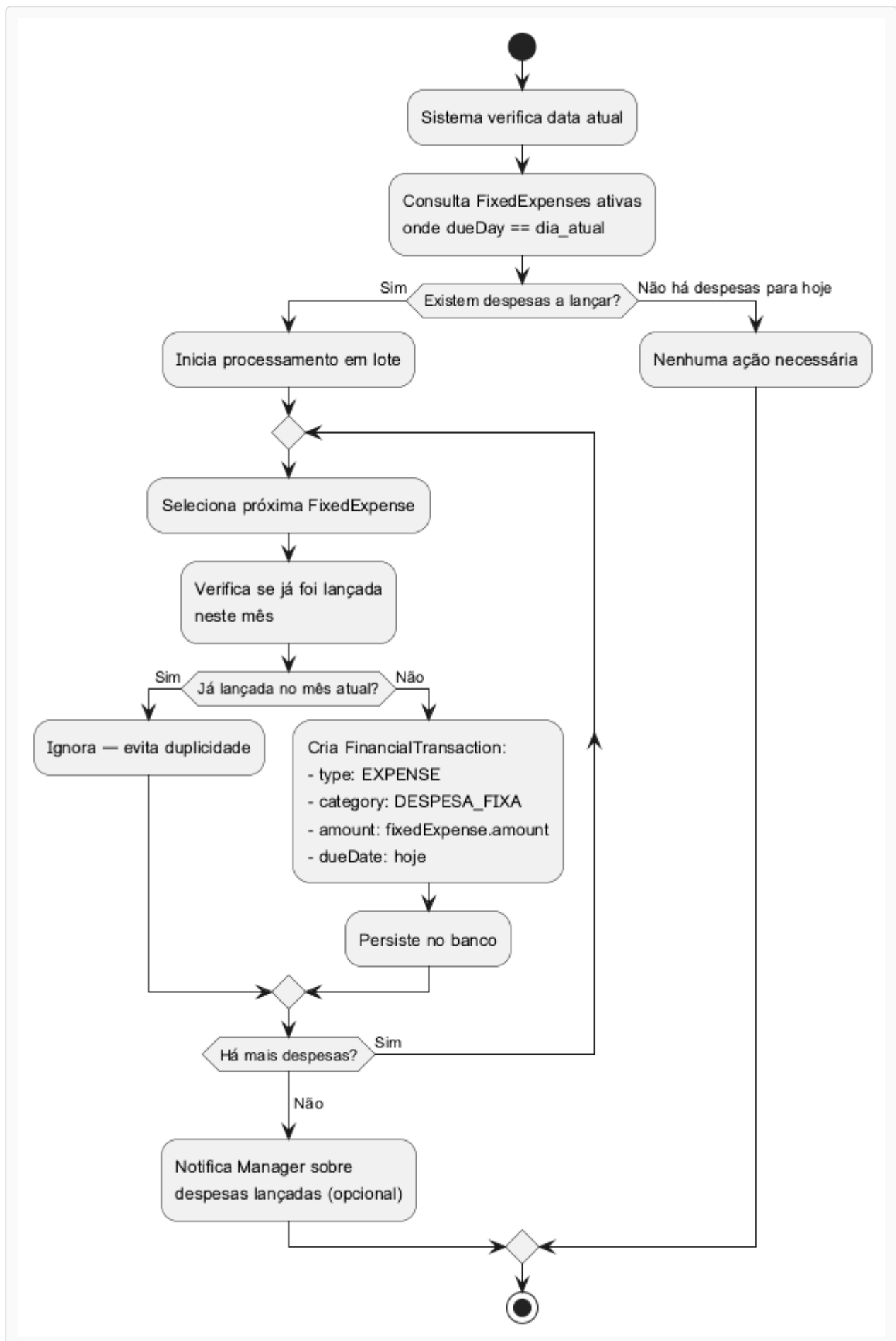


Figura 13 — DA-04: Lançamento de Despesa Fixa Mensal

5. Diagramas de Máquina de Estados

DME-01 — Ciclo de Vida da Ordem de Serviço

O diagrama de estados da OS mapeia todos os estados possíveis (PENDING, WAITING_QUEUE, IN_PROGRESS, READY, COMPLETED, CANCELED), as transições válidas entre eles, as ações de entrada/saída de cada estado e as condições de guarda que controlam as transições.

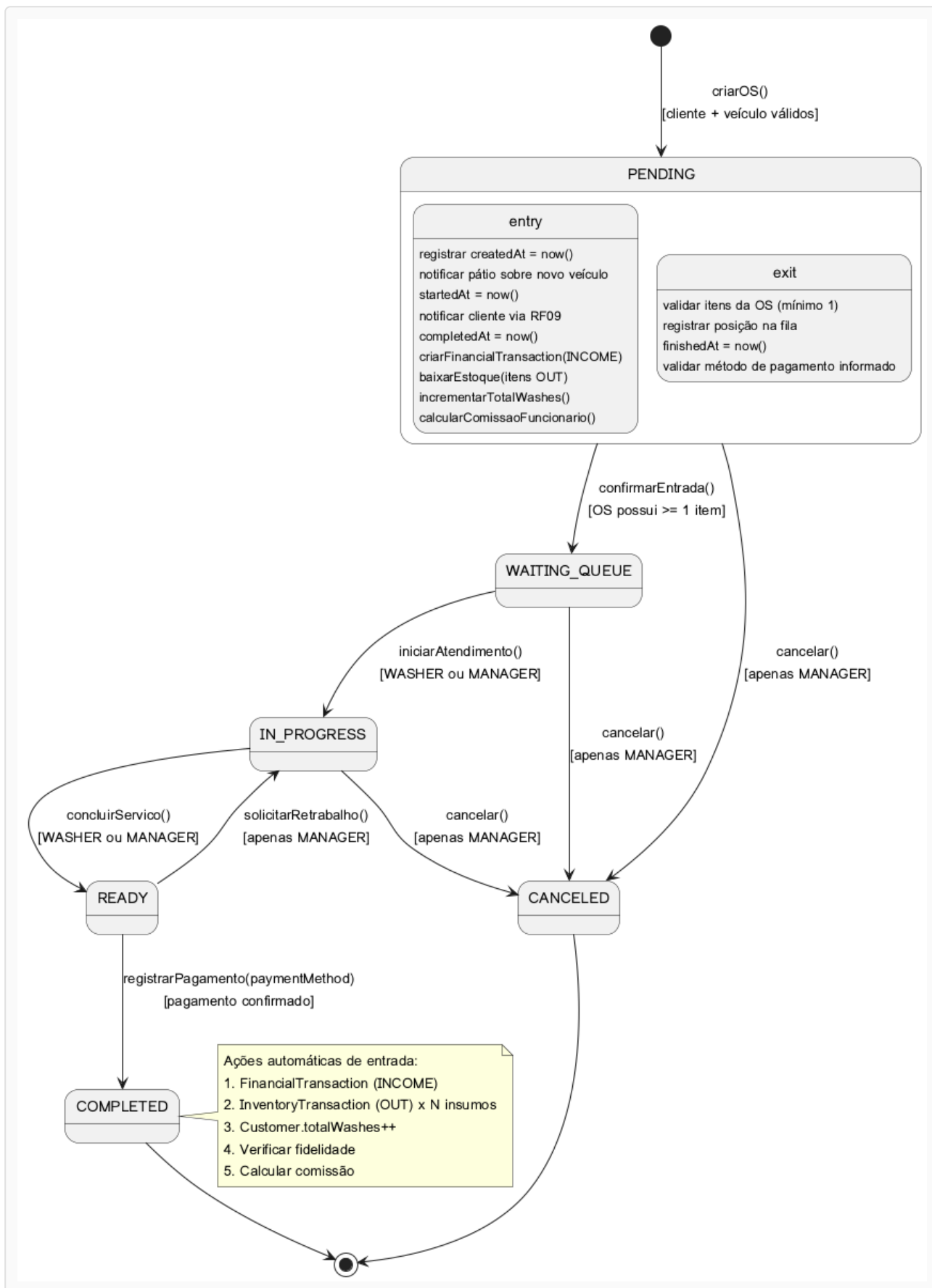


Figura 14 — DME-01: Ciclo de Vida da Ordem de Serviço

DME-02 — Ciclo de Vida do Agendamento

O diagrama de estados do Agendamento mapeia os estados SCHEDULED, COMPLETED e CANCELED, com ênfase na guarda de idempotência que impede a conversão de um agendamento já vinculado a uma OS.

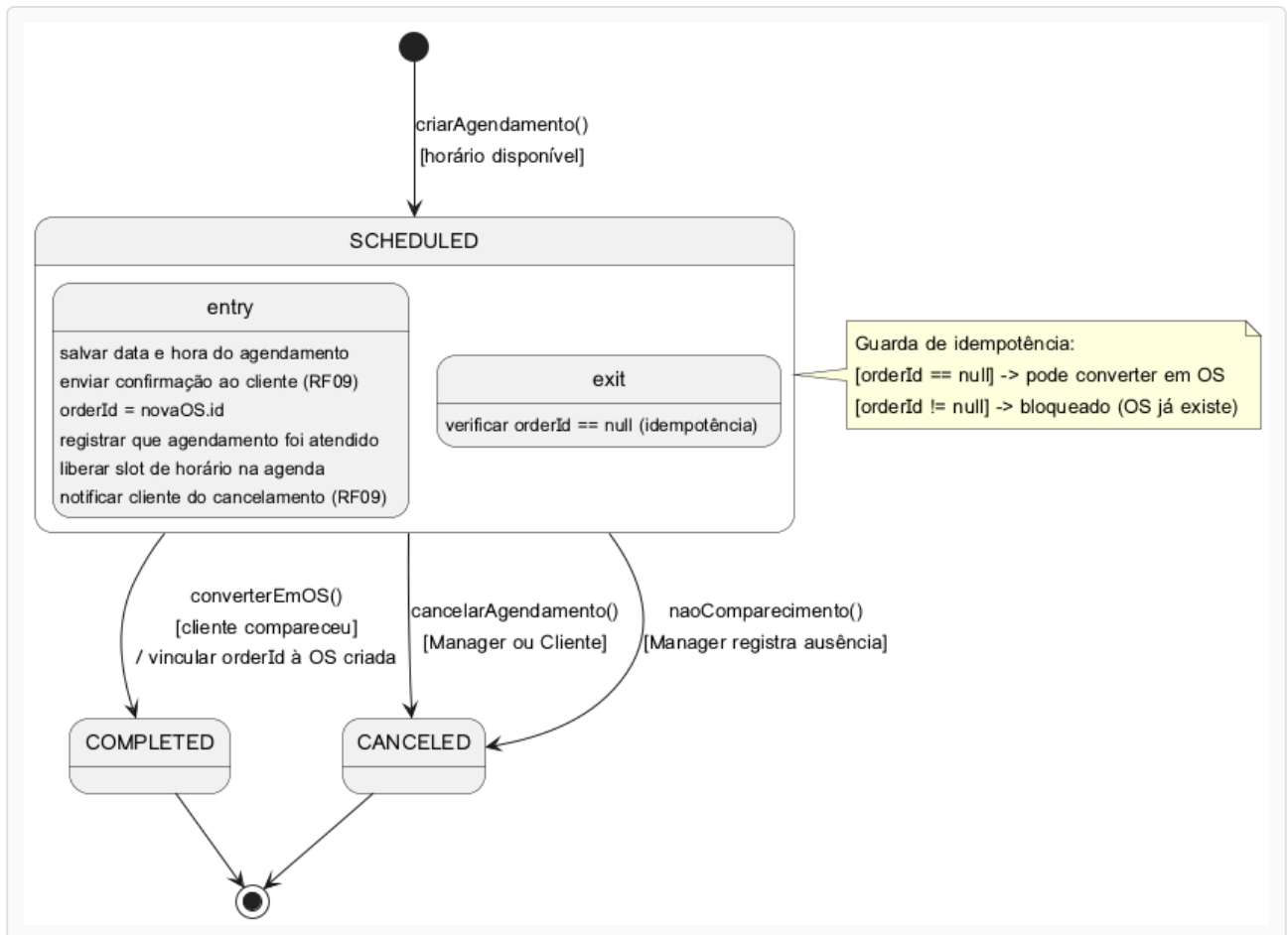


Figura 15 — DME-02: Ciclo de Vida do Agendamento

6. Protótipo

O WashControl possui um protótipo de alta fidelidade funcional desenvolvido em **Next.js 15**. Diferentemente de protótipos estáticos (como Figma), trata-se de uma aplicação web real com banco de dados PostgreSQL, autenticação via NextAuth e Server Actions — tornando o protótipo interativo e demonstrável.

Diagrama de Navegação de Telas

A navegação interliga o Login ao Dashboard e aos módulos específicos, com fluxos de integração entre OS, Agendamentos, Financeiro e Estoque.

Tela	Descrição	Ator Principal
Login	Autenticação com e-mail e senha, redirecionamento por papel	Manager / Washer / SuperAdmin
Dashboard	KPIs: receita do dia/mês, OSs em andamento, meta mensal, gráfico receitas × despesas	Manager
OS — Kanban	Board com 5 colunas de status, drag-and-drop de cards, modal de criação e conclusão de OS	Manager
Pátio	Visão simplificada dos veículos em atendimento, botões de ação para o Washer	Washer
Clientes	CRUD de clientes, veículos vinculados, histórico de lavagens, programa de fidelidade	Manager
Agendamentos	Calendário mensal, formulário de novo agendamento, conversão em OS	Manager
Financeiro	DRE mensal, fluxo de caixa, listagem de transações com filtros	Manager
Insumos	Catálogo de produtos com estoque, alertas de mínimo, histórico de movimentações	Manager
Equipe	Cadastro de funcionários, vinculação com usuário do sistema, comissões	Manager
Configurações	Dados do tenant, configuração de fidelidade, plano SaaS	Manager

7. Item de Criatividade

Os itens abaixo foram identificados como contribuições criativas — itens não solicitados explicitamente pelo enunciado que tornam o trabalho superior:

Criatividade no Projeto (Documento)

1. Sistema real implementado como protótipo de alta fidelidade: Em vez de um protótipo estático no Figma, o WashControl foi desenvolvido como uma aplicação funcional em Next.js 15, com banco de dados real (PostgreSQL), autenticação e Server Actions — tornando o protótipo interativo e demonstrável.

2. Programa de fidelidade parametrizável: A modelagem incluiu campos `loyaltyInterval` e `loyaltyDiscount` no `Tenant`, permitindo que cada lava-car configure seu próprio programa de fidelidade (a cada N lavagens, X% de desconto) — um diferencial competitivo não previsto nos requisitos originais.

3. Técnicas de Elicitação documentadas com rastreabilidade: Cada técnica (Entrevista, Observação, Questionário, Brainstorming, JAD) foi documentada com stakeholder-alvo, roteiro aplicado, limitações encontradas e mapeamento direto para os RFs/RNFs resultantes — vai além da simples listagem de técnicas.

4. Diagrama de Objetos com cenário real e justificativa: O diagrama de objetos apresenta uma instância concreta de atendimento em andamento, com valores reais e justificativa da escolha das classes representadas.

5. Idempotência modelada explicitamente: A restrição `Appointment.orderId @unique` foi especificada como regra de negócio e representada em fluxos alternativos/exceções das especificações de caso de uso e no diagrama de estados do Appointment.

Criatividade no Produto

6. Arquitetura multi-tenant real: O isolamento de dados entre tenants é garantido em nível de banco de dados pelo campo `tenantId` obrigatório em todas as tabelas — não é um isolamento lógico por aplicação, mas estrutural.

7. Automações em cascata ao concluir OS: Ao marcar uma OS como COMPLETED, 5 ações são executadas automaticamente em uma única transação: receita financeira, baixa de

estoque, contagem de fidelidade, cálculo de comissão e verificação de desconto — modelado e implementado.

8. Módulo de faturamento SaaS (B2B): Além do sistema para o lava-car, existe um módulo de faturamento para o Super Admin gerenciar os planos e cobranças dos tenants — tornando o WashControl um produto SaaS completo (produto + plataforma).

8. Responsabilidades

Todos os integrantes da equipe devem assinar e datar o documento de Análise e Projeto do Sistema Computacional.

Integrante	RA	Assinatura	Data
Leonardo dos Santos Marques	—	_____	17/04/2026

Documento elaborado para a disciplina de Modelagem de Sistemas Computacionais — BCC/PUCPR
Sistema: WashControl — Sistema SaaS de Gestão para Lava-Cars e Estéticas Automotivas
©LeonardoDev Inc., 2026